

Task 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
shreyas@Grigori:~$ nano producer_consumer.c
shreyas@Grigori:~$ gcc producer_consumer.c -o producer_consumer -pthread
shreyas@Grigori:~$ ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Produced: 11
Consumed: 8
Produced: 12
Consumed: 9
Produced: 13
Produced: 14
Consumed: 10
Produced: 15
Consumed: 11
Produced: 16
Produced: 16
Consumed: 12
Produced: 17
Consumed: 13
Produced: 18
Consumed: 14
Produced: 19
Consumed: 15
Produced: 20
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20
shreyas@Grigori:~$ sed -i 's/#define BUFFER_SIZE 5/#define BUFFER_SIZE 2/' producer_consumer.c
shreyas@Grigori:~$ gcc producer_consumer.c -o producer_consumer -pthread
shreyas@Grigori:~$ ./producer_consumer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Consumed: 6
```

```
Produced: 7
Consumed: 6
Produced: 8
Consumed: 7
Produced: 9
Consumed: 8
Produced: 10
Consumed: 9
Produced: 11
Consumed: 10
Produced: 12
Consumed: 11
Produced: 13
Consumed: 12
Produced: 14
Consumed: 13
Produced: 15
Consumed: 14
Produced: 16
Consumed: 15
Produced: 17
Consumed: 16
Produced: 18
Consumed: 17
Produced: 19
Consumed: 18
Produced: 20
Consumed: 19
Consumed: 20
```

```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 2
#define ITEMS 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);
```

```

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];

        printf("Consumed: %d\n", item);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

```

```
pthread_create(&consumer_thread, NULL, consumer, NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}
```

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering

```

    {
        printf("Thread 2: Locking Resource 2\n");
        pthread_mutex_lock(&resource2);

        sleep(1);

        printf("Thread 2: Waiting for Resource 1\n");
        pthread_mutex_lock(&resource1);

        printf("Thread 2: Acquired both resources\n");

        pthread_mutex_unlock(&resource1);
        pthread_mutex_unlock(&resource2);

        return NULL;
    }

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}

```

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)

```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
./deadlock_prevention
```

```
Thread 1: Locking Resource 2
```

```
Thread 1: Acquired both resources
```

```
Thread 2: Locking Resource 2
```

```
Thread 2: Acquired both resources
```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
Thread 1: Locking Resource 2
```

```
Thread 1: Acquired both resources
```

```
Thread 2: Locking Resource 2
```

```
Thread 2: Acquired both resources
```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
Thread 1: Locking Resource 2
```

```
Thread 1: Acquired both resources
```

```
Thread 2: Locking Resource 2
```

```
Thread 2: Acquired both resources
```

```
shreyas@Grigori:~$ |
```

```
shreyas@Grigori:~$ nano producer_consumer.c
```

```
shreyas@Grigori:~$ nano deadlock.c
```

```
shreyas@Grigori:~$ gcc deadlock.c -o deadlock -pthread
```

```
shreyas@Grigori:~$ ./deadlock
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 2
```

```
Thread 1: Waiting for Resource 2
```

```
Thread 2: Waiting for Resource 1
```

```
^C
```

```
shreyas@Grigori:~$ nano deadlock_prevention.c
```

```
shreyas@Grigori:~$ gcc deadlock_prevention.c -o deadlock_prevention -pthread
```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
Thread 1: Locking Resource 2
```

```
Thread 1: Acquired both resources
```

```
Thread 2: Locking Resource 2
```

```
Thread 2: Acquired both resources
```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
./deadlock_prevention
```

```
Thread 1: Locking Resource 2
```

```
Thread 1: Acquired both resources
```

```
Thread 2: Locking Resource 2
```

```
Thread 2: Acquired both resources
```

```
shreyas@Grigori:~$ ./deadlock_prevention
```

```
Thread 1: Locking Resource 1
```

```
Thread 2: Locking Resource 1
```

```
pthread_join(t2, NULL);

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

return 0;
}
```

```
GNU nano 7.2 deadlock_prevent.c
void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
```